CO3_Project_DelaCruz_Morales

July 27, 2025

Dela Cruz, Cloie Sofia R.

Morales, Rayleen Claire G.

AM1

1 Part A: Data Acquisition & Preprocessing

Data Source Research

Dataset Source: Kaggle - Brazilian E-Commerce Public Dataset by Olist

https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce?select=olist_customers_dataset.csv&fbclid=IwY
VU7p0IcqP0_Mf_aem_x0OHd_miQCcAKpWb_Gx9Gw

Data Loading & Initial Exploration

Import Libraries

[56]:
```
# This line of code will import the pandas library into the jupyter notebook.
# Pandas is a python package that we will use to analyse and manipulate our␣
 ↪dataset.
# Using the 'as' syntax, we were able to assign the alias 'pd' to panda to␣
 ↪reference its functions easier throughout the script.
import pandas as pd

# This line of code will import the numpy library.
# NumPy is a library in Python for the computation and manipulation of data.
# Using the 'as' syntax, we were able to assign the alias 'np' to numpy to␣
 ↪import numpy as np
import matplotlib.pyplot as plt

# This line of code will import the Seaborn library.
# Seaborn is a Phyton library used for visualization built on top of Matplotlib.
# Using 'sns,' we have given it an alias for shorthand use throughout our code.
import seaborn as sns

# This line of code imports the PCA (Principal Component Analysis) class from␣
 ↪scikit-learn's decomposition module.
```

```
# PCA will be used to reduce customer feature space to two dimensions for easy␣
 ↪visualization of clusters.
from sklearn.decomposition import PCA

# This line of code imports the Davies-Bouldin Index evaluation function from␣
 ↪scikit-learn's metrics module.
# The Davies-Bouldin Index measures the average similarity between clusters,␣
 ↪with lower values indicating better clustering performance.
from sklearn.metrics import davies_bouldin_score

# This imports the KMeans clustering algorithm from scikit-learn's cluster␣
 ↪module.
# KMeans is an unsupervised learning algorithm that partitions data into k␣
 ↪distinct clusters by minimizing within-cluster variance.
from sklearn.cluster import KMeans

# This line of code imports the StandardScaler class from scikit-learn's␣
 ↪preprocessing module.
# This class will help us standardize features by removing the mean and scaling␣
 ↪to unit variance.
from sklearn.preprocessing import StandardScaler

# This imports the Silhouette Score evaluation function from scikit-learn's␣
 ↪metrics module.
# It will help determine the quality of clustering and supports selecting the␣
 ↪optimal number of clusters.
from sklearn.metrics import silhouette_score

# MLxtend (Machine learning extensions) is a Python library that is used for␣
 ↪data science tasks (Raschka, 2014).
# This import will be used for the market basket analysis.
from mlxtend.frequent_patterns import apriori, association_rules
```

Load CSV Files

[57]:
```
# This line of code reads the file olist_orders_dataset.csv into a Pandas␣
 ↪DataFrame called orders.
# It loads order-level data, which typically includes order IDs, customer IDs,␣
 ↪purchase timestamps, and delivery details.
orders = pd.read_csv('olist_orders_dataset.csv')

# This reads the olist_order_items_dataset.csv file into a DataFrame named␣
 ↪items.
# This dataset is essential for understanding what specific products and␣
 ↪quantities are included in each order.
items = pd.read_csv('olist_order_items_dataset.csv')
```

```
# This line of code loads the olist_products_dataset.csv file into a DataFrame␣
 ↪called products.
# It contains produce-level details we need such as product category,␣
 ↪dimensions, weight, and other descriptive information.
products = pd.read_csv('olist_products_dataset.csv')

# This reads the product_category_name_translation.csv file into a DataFrame␣
 ↪names product_cat.
# It contains a mapping of product category names from Portuguese to English.
product_cat = pd.read_csv('product_category_name_translation.csv')

# This loads the olist_customers_dataset.csv file into a DataFrame named␣
 ↪customers.
# This dataset is important for associating orders with individual customers␣
 ↪and performing customer-based analysis.
customers = pd.read_csv('olist_customers_dataset.csv')
```

Initial Data Exploration

[58]:
```
# This line defines a Python function named explore_dataframe that takes a␣
 ↪DataFrame df as input.
# The function is designed to quickly explore and summarize the structure and␣
 ↪quality of the dataset.
def explore_dataframe(df, name="DataFrame"):

    # This prints a header indicating which dataset is being explored.
    # It uses an f-string to dynamically insert the dataset's name.
    print(f"--- Exploration for {name} ---")

    # Prints a label for better readability for the output sections.
    # This indicates that the following printed values relates to dataset␣
 ↪dimensions.
    print("\n1. Shape:")

    # This prints the shape of the DataFrame as a tuple.
    # It tells us how many records (rows) and attributes (columns) the dataset␣
 ↪contains.
    print(df.shape)

    # This prints a label indicating that the next output will how the first 5␣
 ↪rows.
    # This will help us focus on what part of the dataset is being displayed.
    print("\n2. First 5 rows:")

    # This line uses Pandas' head() method to display the first five rows of␣
 ↪the dataset.
```

```
    # The display() will be used in our notebook to render a richer output than␣
↪print().
    display(df.head())

    # This line prints a label indicating that the dataset information summary␣
↪text is next.
    # It separates this section from the previous one.
    print("\n3. Info:")

    # This line of code prints a concise summary of the dataset, including␣
↪column names, non-null counts, and data types.
    # It helps in identifying missing values and understand memory usage.
    print(df.info())

    # This prints a label indicating that descriptive statistics will follow.
    # This helps differentiating this section from others for easy navigation.
    print("\n4. Statistical Summary (numeric columns):")

    # This line generates descriptive statistics (mean, std, min, max,␣
↪quartiles) for numeric columns.
    # This is valuable for spotting outliers and distribution ranges.
    display(df.describe())

    # This line prints a label for the missing values check.
    # It will help us quickly find data quality issues.
    print("\n5. Missing Values Count:")

    # This line of code calculates how many missing values each column has␣
↪using isnull().sum().
    # This will help determine which columns need cleaning or imputation.
    print(df.isnull().sum())

    # This line prints a label for the duplicate rows check.
    # It will separate duplicate detection from missing value detection for␣
↪clarity.
    print("\n6. Duplicated Rows Count:")

    # This line counts how many duplicate rows are present in the dataset using␣
↪duplicated(). sum()
    # This quick check ensures data integrity.
    print(df.duplicated().sum())

    # This line prints a label for data type information.
    # This will help quickly verify if numeric, categorical, or datetime data␣
↪type are correctly set.
    print("\n7. Column Data Types:")
```

```
    # This line displays the data type of each column.
    # This will be useful for spotting wrong data types, like numbers stored as␣
 ↪strings.
    print(df.dtypes)

    # This line prints a separator line made of 50 dashes.
    # It visually separates the summary of one dataset from the next.
    print("-" * 50)
```

[59]:
```
# This line calls the explore_dataframe() with the orders dataset and names it␣
 ↪"Orders."
# This prints the summary information of the orders dataset.
explore_dataframe(orders, "Orders")
```

```
--- Exploration for Orders ---

1. Shape:
(99441, 8)

2. First 5 rows:

                           order_id                       customer_id  \
0  e481f51cbdc54678b7cc49136f2d6af7  9ef432eb6251297304e76186b10a928d
1  53cdb2fc8bc7dce0b6741e2150273451  b0830fb4747a6c6d20dea0b8c802d7ef
2  47770eb9100c2d0c44946d9cf07ec65d  41ce2a54c0b03bf3443c3d931a367089
3  949d5b44dbf5de918fe9c16f97b45f8a  f88197465ea7920adcdbec7375364d82
4  ad21c59c0840e6cb83a9ceb5573f8159  8ab97904e6daea8866dbdbc4fb7aad2c

  order_status order_purchase_timestamp    order_approved_at  \
0    delivered      2017-10-02 10:56:33  2017-10-02 11:07:15
1    delivered      2018-07-24 20:41:37  2018-07-26 03:24:27
2    delivered      2018-08-08 08:38:49  2018-08-08 08:55:23
3    delivered      2017-11-18 19:28:06  2017-11-18 19:45:59
4    delivered      2018-02-13 21:18:39  2018-02-13 22:20:29

  order_delivered_carrier_date order_delivered_customer_date  \
0          2017-10-04 19:55:00           2017-10-10 21:25:13
1          2018-07-26 14:31:00           2018-08-07 15:27:45
2          2018-08-08 13:50:00           2018-08-17 18:06:29
3          2017-11-22 13:39:59           2017-12-02 00:28:42
4          2018-02-14 19:46:34           2018-02-16 18:17:02

  order_estimated_delivery_date
0           2017-10-18 00:00:00
1           2018-08-13 00:00:00
2           2018-09-04 00:00:00
3           2017-12-15 00:00:00
```

```
4           2018-02-26 00:00:00


3. Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99441 entries, 0 to 99440
Data columns (total 8 columns):
 #   Column                         Non-Null Count  Dtype
---  ------                         --------------  -----
 0   order_id                       99441 non-null  object
 1   customer_id                    99441 non-null  object
 2   order_status                   99441 non-null  object
 3   order_purchase_timestamp       99441 non-null  object
 4   order_approved_at              99281 non-null  object
 5   order_delivered_carrier_date   97658 non-null  object
 6   order_delivered_customer_date  96476 non-null  object
 7   order_estimated_delivery_date  99441 non-null  object
dtypes: object(8)
memory usage: 6.1+ MB
None

4. Statistical Summary (numeric columns):

                                order_id                         customer_id  \
count                              99441                               99441
unique                             99441                               99441
top     e481f51cbdc54678b7cc49136f2d6af7  9ef432eb6251297304e76186b10a928d
freq                                   1                                   1

       order_status order_purchase_timestamp    order_approved_at  \
count         99441                    99441                99281
unique            8                    98875                90733
top       delivered      2018-08-02 12:06:07  2018-02-27 04:31:10
freq          96478                        3                    9

       order_delivered_carrier_date order_delivered_customer_date  \
count                         97658                         96476
unique                        81018                         95664
top             2018-05-09 15:48:00           2018-05-14 20:02:44
freq                             47                             3

       order_estimated_delivery_date
count                          99441
unique                           459
top              2017-12-20 00:00:00
freq                             522


5. Missing Values Count:
```

```
order_id                            0
customer_id                         0
order_status                        0
order_purchase_timestamp            0
order_approved_at                 160
order_delivered_carrier_date     1783
order_delivered_customer_date    2965
order_estimated_delivery_date       0
dtype: int64

6. Duplicated Rows Count:
0

7. Column Data Types:
order_id                         object
customer_id                      object
order_status                     object
order_purchase_timestamp         object
order_approved_at                object
order_delivered_carrier_date     object
order_delivered_customer_date    object
order_estimated_delivery_date    object
dtype: object
--------------------------------------------------
```

[60]:
```
# This line explores the items dataset, showing its shape, missing values, and column types.
# This dataset details each item in every order.
explore_dataframe(items, "Items")
```

```
--- Exploration for Items ---

1. Shape:
(112650, 7)

2. First 5 rows:
                           order_id  order_item_id  \
0  00010242fe8c5a6d1ba2dd792cb16214              1
1  00018f77f2f0320c557190d7a144bdd3              1
2  000229ec398224ef6ca0657da4fc703e              1
3  00024acbcdf0a6daa1e931b038114c75              1
4  00042b26cf59d7ce69dfabb4e55b4fd9              1

                         product_id                         seller_id  \
0  4244733e06e7ecb4970a6e2683c13e61  48436dade18ac8b2bce089ec2a041202
1  e5f2d52b802189ee658865ca93d83a8f  dd7ddc04e1b6c2c614352b383efe2d36
2  c777355d18b72b67abbeef9df44fd0fd  5b51032eddd242adc84c38acab88f23d
```

```
3  7634da152a4610f1595efa32f14722fc  9d7a1d34a5052409006425275ba1c2b4
4  ac6c3623068f30de03045865e4e10089  df560393f3a51e74553ab94004ba5c87

   shipping_limit_date   price  freight_value
0  2017-09-19 09:45:35   58.90          13.29
1  2017-05-03 11:05:13  239.90          19.93
2  2018-01-18 14:48:30  199.00          17.87
3  2018-08-15 10:10:18   12.99          12.79
4  2017-02-13 13:57:51  199.90          18.14


3. Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 112650 entries, 0 to 112649
Data columns (total 7 columns):
 #   Column               Non-Null Count   Dtype
---  ------               --------------   -----
 0   order_id             112650 non-null  object
 1   order_item_id        112650 non-null  int64
 2   product_id           112650 non-null  object
 3   seller_id            112650 non-null  object
 4   shipping_limit_date  112650 non-null  object
 5   price                112650 non-null  float64
 6   freight_value        112650 non-null  float64
dtypes: float64(2), int64(1), object(4)
memory usage: 6.0+ MB
None

4. Statistical Summary (numeric columns):

       order_item_id          price  freight_value
count  112650.000000  112650.000000  112650.000000
mean        1.197834     120.653739      19.990320
std         0.705124     183.633928      15.806405
min         1.000000       0.850000       0.000000
25%         1.000000      39.900000      13.080000
50%         1.000000      74.990000      16.260000
75%         1.000000     134.900000      21.150000
max        21.000000    6735.000000     409.680000


5. Missing Values Count:
order_id              0
order_item_id         0
product_id            0
seller_id             0
shipping_limit_date   0
price                 0
freight_value         0
```

```
dtype: int64

6. Duplicated Rows Count:
0

7. Column Data Types:
order_id                object
order_item_id            int64
product_id              object
seller_id               object
shipping_limit_date     object
price                  float64
freight_value          float64
dtype: object
--------------------------------------------------
```

[61]:
```
# This line explores the products dataset to show product-level information.
# This includes attributes such as dimensions, weight, and category name.
explore_dataframe(products, "Products")
```

```
--- Exploration for Products ---

1. Shape:
(32951, 9)

2. First 5 rows:
                         product_id  product_category_name  \
0  1e9e8ef04dbcff4541ed26657ea517e5             perfumaria
1  3aa071139cb16b67ca9e5dea641aaa2f                  artes
2  96bd76ec8810374ed1b65e291975717f          esporte_lazer
3  cef67bcfe19066a932b7673e239eb23d                  bebes
4  9dc1a7de274444849c219cff195d0b71  utilidades_domesticas

   product_name_lenght  product_description_lenght  product_photos_qty  \
0                 40.0                       287.0                 1.0
1                 44.0                       276.0                 1.0
2                 46.0                       250.0                 1.0
3                 27.0                       261.0                 1.0
4                 37.0                       402.0                 4.0

   product_weight_g  product_length_cm  product_height_cm  product_width_cm
0             225.0               16.0               10.0              14.0
1            1000.0               30.0               18.0              20.0
2             154.0               18.0                9.0              15.0
3             371.0               26.0                4.0              26.0
4             625.0               20.0               17.0              13.0
```

```
3. Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32951 entries, 0 to 32950
Data columns (total 9 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   product_id                  32951 non-null  object
 1   product_category_name       32341 non-null  object
 2   product_name_lenght         32341 non-null  float64
 3   product_description_lenght  32341 non-null  float64
 4   product_photos_qty          32341 non-null  float64
 5   product_weight_g            32949 non-null  float64
 6   product_length_cm           32949 non-null  float64
 7   product_height_cm           32949 non-null  float64
 8   product_width_cm            32949 non-null  float64
dtypes: float64(7), object(2)
memory usage: 2.3+ MB
None

4. Statistical Summary (numeric columns):

       product_name_lenght  product_description_lenght  product_photos_qty  \
count         32341.000000                32341.000000        32341.000000
mean             48.476949                  771.495285            2.188986
std              10.245741                  635.115225            1.736766
min               5.000000                    4.000000            1.000000
25%              42.000000                  339.000000            1.000000
50%              51.000000                  595.000000            1.000000
75%              57.000000                  972.000000            3.000000
max              76.000000                 3992.000000           20.000000

       product_weight_g  product_length_cm  product_height_cm  \
count      32949.000000       32949.000000       32949.000000
mean        2276.472488          30.815078          16.937661
std         4282.038731          16.914458          13.637554
min            0.000000           7.000000           2.000000
25%          300.000000          18.000000           8.000000
50%          700.000000          25.000000          13.000000
75%         1900.000000          38.000000          21.000000
max        40425.000000         105.000000         105.000000

       product_width_cm
count      32949.000000
mean          23.196728
std           12.079047
min            6.000000
25%           15.000000
50%           20.000000
```

```
75%           30.000000
max          118.000000


5. Missing Values Count:
product_id                      0
product_category_name         610
product_name_lenght           610
product_description_lenght    610
product_photos_qty            610
product_weight_g                2
product_length_cm               2
product_height_cm               2
product_width_cm                2
dtype: int64

6. Duplicated Rows Count:
0

7. Column Data Types:
product_id                     object
product_category_name          object
product_name_lenght           float64
product_description_lenght    float64
product_photos_qty            float64
product_weight_g              float64
product_length_cm             float64
product_height_cm             float64
product_width_cm              float64
dtype: object
--------------------------------------------------
```

[62]:
```
# This line explores the product category translation dataset.
# It helps confirm if all category names are translated without missing values.
explore_dataframe(product_cat, "Product Categories")
```

```
--- Exploration for Product Categories ---

1. Shape:
(71, 2)

2. First 5 rows:
    product_category_name product_category_name_english
0            beleza_saude                  health_beauty
1  informatica_acessorios         computers_accessories
2              automotivo                          auto
3         cama_mesa_banho                bed_bath_table
4        moveis_decoracao               furniture_decor
```

```
3. Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 71 entries, 0 to 70
Data columns (total 2 columns):
 #   Column                         Non-Null Count  Dtype
---  ------                         --------------  -----
 0   product_category_name          71 non-null     object
 1   product_category_name_english  71 non-null     object
dtypes: object(2)
memory usage: 1.2+ KB
None

4. Statistical Summary (numeric columns):
       product_category_name product_category_name_english
count                     71                           71
unique                    71                           71
top             beleza_saude                health_beauty
freq                       1                            1


5. Missing Values Count:
product_category_name            0
product_category_name_english    0
dtype: int64

6. Duplicated Rows Count:
0

7. Column Data Types:
product_category_name            object
product_category_name_english    object
dtype: object
--------------------------------------------------
```

[63]:
```
# This line explores the customer dataset.
# It helps verify if there are duplicate or incomplete customer records.
explore_dataframe(customers, "Customers")
```

```
--- Exploration for Customers ---

1. Shape:
(99441, 5)

2. First 5 rows:
                        customer_id                customer_unique_id  \
0  06b8999e2fba1a1fbc88172c00ba8bc7  861eff4711a542e4b93843c6dd7febb0
1  18955e83d337fd6b2def6b18a428ac77  290c77bc529b7ac935b93aa66c333dc3
```

```
2  4e7b3e00288586ebd08712fdd0374a03  060e732b5b29e8181a18229c7b0b2b5e
3  b2b6027bc5c5109e529d4dc6358b12c3  259dac757896d24d7702b9acbbff3f3c
4  4f2d8ab171c80ec8364f7c12e35b23ad  345ecd01c38d18a9036ed96c73b8d066

   customer_zip_code_prefix          customer_city customer_state
0                     14409                 franca             SP
1                      9790  sao bernardo do campo             SP
2                      1151              sao paulo             SP
3                      8775        mogi das cruzes             SP
4                     13056               campinas             SP


3. Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99441 entries, 0 to 99440
Data columns (total 5 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   customer_id              99441 non-null  object
 1   customer_unique_id       99441 non-null  object
 2   customer_zip_code_prefix  99441 non-null  int64
 3   customer_city            99441 non-null  object
 4   customer_state           99441 non-null  object
dtypes: int64(1), object(4)
memory usage: 3.8+ MB
None

4. Statistical Summary (numeric columns):

       customer_zip_code_prefix
count              99441.000000
mean               35137.474583
std                29797.938996
min                 1003.000000
25%                11347.000000
50%                24416.000000
75%                58900.000000
max                99990.000000


5. Missing Values Count:
customer_id                 0
customer_unique_id          0
customer_zip_code_prefix    0
customer_city               0
customer_state              0
dtype: int64

6. Duplicated Rows Count:
```

```
0

7. Column Data Types:
customer_id                 object
customer_unique_id          object
customer_zip_code_prefix     int64
customer_city               object
customer_state              object
dtype: object
--------------------------------------------------
```

Data Preprocessing

Clustering Preprocessing

[64]:
```
# This line of code merges the items dataset with the orders dataset to attach␣
 ↪customer IDs to each order item.
# on='order_id' ensures the merge is based on matching order IDs.
# how='left' keeps all rows from items, ensuring no loss of order-item␣
 ↪relationships even if some orders are missing customer information.
order_items_merged = pd.merge(items, orders[['order_id', 'customer_id']],␣
 ↪on='order_id', how='left')

# This line adds customer details like unique ID, city, and state to each␣
 ↪transaction.
# This results in each row having both product-level and customer-level␣
 ↪information.
customer_orders = pd.merge(order_items_merged, customers[['customer_id',␣
 ↪'customer_unique_id', 'customer_city', 'customer_state']], on='customer_id',␣
 ↪how='left')

# This line attaches product category information to each item purchased.
# This allows later analysis of what kinds of products customers are buying.
customer_orders = pd.merge(customer_orders, products[['product_id',␣
 ↪'product_category_name']], on='product_id', how='left')

# Portuguese category names to English using the translation dataset.
# This ensures all product categories are easily interpretable for reporting.
customer_orders = pd.merge(customer_orders, product_cat,␣
 ↪on='product_category_name', how='left')

# This line fills any missing price values with 0.
# Ensures that sum calculations later do not fail or produce NaN results.
customer_orders['price'] = customer_orders['price'].fillna(0)

# This line of code fills missing product category translations with "unknown".
# This ensures all products have some category label for counting unique␣
 ↪categories.
```

```
customer_orders['product_category_name_english'] = \
    customer_orders['product_category_name_english'].fillna('unknown')

# This line groups all transactions by each unique customer ID.
# It calculates aggregated features: number of orders, total spend, unique␣
 ↪categories, and total items bought and produces one summarized row for each␣
 ↪customer.
customer_features = customer_orders.groupby('customer_unique_id').agg({
    'order_id': 'nunique',                      # number of unique orders
    'price': 'sum',                             # total spend
    'product_category_name_english': 'nunique', # number of unique categories
    'order_item_id': 'count'                    # total items purchased

    # This renames the columns for clarity after aggregation.
    # It makes clear which features represent counts, sums, or unique values.
}).rename(columns={
    'order_id': 'num_orders',
    'price': 'total_price',
    'product_category_name_english': 'unique_categories',
    'order_item_id': 'total_items'
})

# This line creates a new feature representing how much a customer spends on␣
 ↪average per order.
# It divides total spending by the number of unique orders for that customer.
# This helps distinguish between frequent small buyers and infrequent big␣
 ↪spenders.
customer_features['avg_spend_per_order'] = \
    customer_features['total_price'] / customer_features['num_orders']

# This line creates another derived feature for average basket size per order.
# It divides total items purchased by the number of orders.
# This indicates how many products customers typically buy in a single order.
customer_features['avg_basket_size'] = \
    customer_features['total_items'] / customer_features['num_orders']

# This line of code calculates the 99th percentile value of total spending.
# This value acts as a threshold to detect extreme spenders and ensures that␣
 ↪only the top 1% are treated as outliers.
cap_value = customer_features['total_price'].quantile(0.99)

# This caps all spending values above the 99th percentile at the threshold.
# This prevents very high spenders from skewing cluster centroids.
customer_features['total_price'] = \
    customer_features['total_price'].clip(upper=cap_value)

# This line creates a list of column names to use as clustering features.
```

```
# These features capture key aspects of customer behavior: frequency, spending,␣
 ↪diversity, and basket size.
features_for_clustering = ['num_orders', 'total_price', 'unique_categories',␣
 ↪'avg_spend_per_order', 'avg_basket_size']

# This line creates a StandardScaler object from scikit-learn.
# This scaler transforms data to have mean = 0 and standard deviation = 1.
scaler = StandardScaler()

# This line of code applies scaling to the selected feature columns.
# fit_transform calculates scaling parameters (mean, std) and applies them.
X_scaled = scaler.fit_transform(customer_features[features_for_clustering])

# This line prints the shape of the scaled feature matrix.
# It confirms how many customers and features are in the final dataset.
print("Shape of final feature matrix:", X_scaled.shape)

# This line displays the first few rows of customer_features.
# It helps verify that new derived features and capped values are present.
customer_features.head()
```

```
Shape of final feature matrix: (95420, 5)
```

[64]:

customer_unique_id	num_orders	total_price	unique_categories	total_items	avg_spend_per_order	avg_basket_size
0000366f3b9a7992bf8c76cfdf3221e2	1	129.90	1	1	129.90	1.0
0000b849f77a49e4a4ce2b2a4ca5be3f	1	18.90	1	1	18.90	1.0
0000f46a3911fa3c0805444483337064	1	69.00	1	1	69.00	1.0
0000f6ccb0745a6a4b88665a16c9f078	1	25.99	1	1	25.99	1.0
0004aac84e0df4da2b147fca70cf8255	1	180.00	1	1	180.00	1.0

Association Rule Preprocessing

[65]:
```
# This line merges items with products based on the product_id column.
# This attaches product details, such as category, to each order item.
# The how='left' ensures all order items are preserved even if some products␣
 ↪are missing information.
basket_data = items.merge(products, on='product_id', how='left')

# This line adds English translations of product category names using the␣
 ↪product_cat table.
# This enables easier interpretation for non-Portuguese speakers.
basket_data = basket_data.merge(product_cat, on='product_category_name',␣
 ↪how='left')

# This line of code creates a new column product_name that prefers the English␣
 ↪category name if available.
# If the English translation is missing, it falls back to the original category␣
 ↪name.
basket_data['product_name'] = basket_data['product_category_name_english'].
 ↪fillna(basket_data['product_category_name'])

# This line groups products by order and counts how many times each product␣
 ↪appears.
# unstack() pivots product names into columns, creating a basket format where␣
 ↪each column is a product.
# fillna(0) replaces missing values with 0, indicating no purchase of that␣
 ↪product in a given order.
basket = basket_data.groupby(['order_id', 'product_name'])['order_item_id'].
 ↪count().unstack().fillna(0)

# This line converts counts into binary values (1 if purchased, 0 if not).
# gt(0) returns True for counts greater than 0, then .astype(int) converts True/
 ↪False into 1/0.
basket = basket.gt(0).astype(int)
```

2 Part B: Customer Segmentation Using Clustering

Research on Clustering Algorithms

Discussion is found on separate PDF.

Model Building & Evaluation

Code

[66]:
```
# This line of code assigns the previously scaled customer feature matrix to X.
# X_scaled contains normalized numeric features used for KMeans clustering.
X = X_scaled
```

```
# This line initializes an empty list sse to store Sum of Squared Errors␣
 ↪(inertia).
# Each loop iteration will append inertia for a specific cluster count.
sse = []

# This loops through possible cluster counts from 2 to 10.
# It starts at 2 because at least two clusters are needed for meaningful␣
 ↪separation.
for k in range(2, 11):

    # This line creates a KMeans clustering model with k clusters.
    # random_state=42 ensures reproducibility of results.
    kmeans = KMeans(n_clusters=k, random_state=42)

    # This line fits the KMeans model to the scaled feature dataset.
    # This assigns data points to clusters and computes cluster centroids.
    # It also calculates inertia (SSE) for this clustering configuration.
    kmeans.fit(X)

    # This stores the inertia value from the fitted KMeans model in the sse␣
 ↪list.
    # Inertia measures how internally compact the clusters are (lower is␣
 ↪better).
    sse.append(kmeans.inertia_)

# This line plots the number of clusters versus the inertia values.
# marker='o' ensures each point on the curve is clearly marked.
plt.plot(range(2, 11), sse, marker='o')

# This adds a label to the x-axis of the plot.
# It indicates that the x-axis values represent different cluster counts.
plt.xlabel('Number of clusters')

# This line labels the y-axis with "SSE (Inertia)".
# It clarifies that the plotted values measure cluster compactness.
plt.ylabel('SSE (Inertia)')

# This adds a title to the elbow plot.
# It clearly states the purpose of the plot.
plt.title('Elbow Method for Optimal k')

# This displays the elbow plot in the notebook output.
# It ensures that all previous plot commands render a final image.
plt.show()

# This line introduces silhouette scoring to evaluate cluster quality.
# This is an intrinsic metric for unsupervised clustering validation.
```

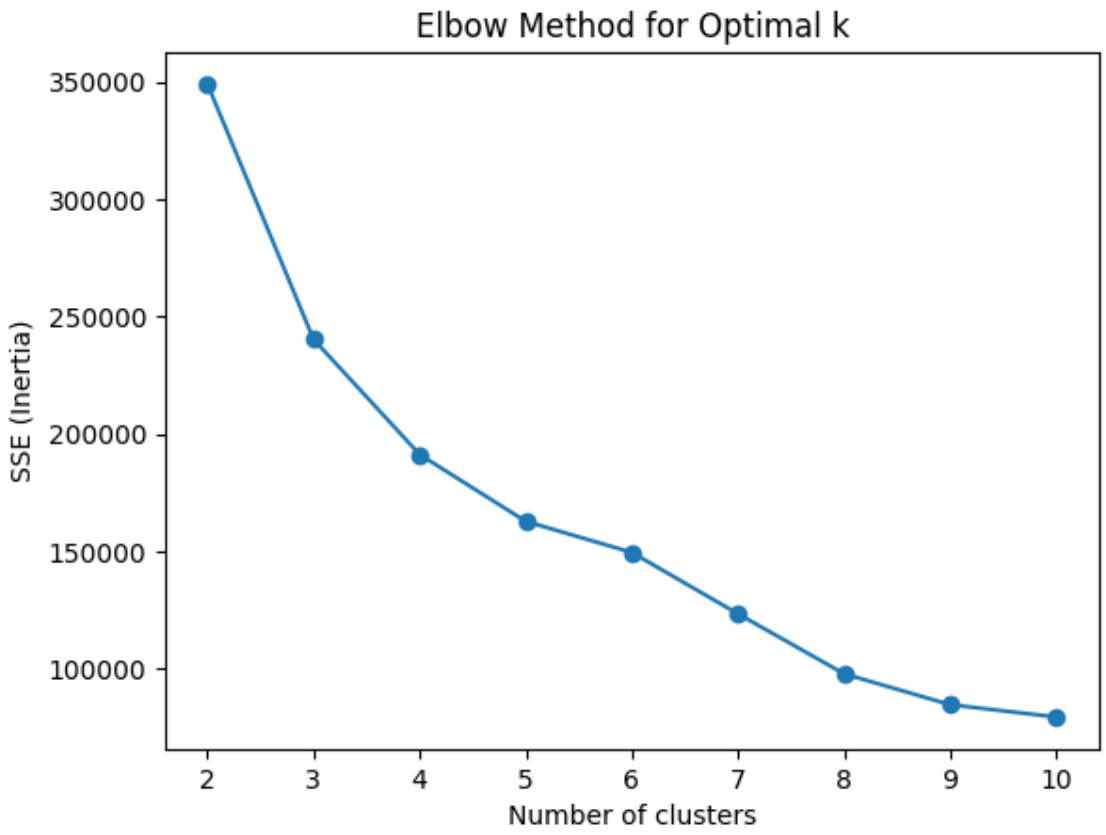
```
# It loops through cluster counts from 2 to 6.
# We focused on a smaller range because silhouette scoring can be␣
 ↪computationally heavy.
for k in range(2, 7):

    # This creates and immediately fits a KMeans model for k clusters.
    # random_state=42 ensures consistent clustering results each run.
    # After fitting, the model produces cluster assignments.
    kmeans = KMeans(n_clusters=k, random_state=42).fit(X)

    # This line retrieves the cluster labels assigned to each data point.
    # These labels indicate which cluster each observation belongs to.
    labels = kmeans.labels_

    # This line of code calculates the silhouette score based on feature data␣
 ↪and assigned labels.
    # Higher scores mean better separation and cohesion of clusters.
    score = silhouette_score(X, labels)

    # This line prints the number of clusters and its corresponding silhouette␣
 ↪score.
    # :.3f formats the score to three decimal places for readability.
    print(f"k={k}, silhouette score={score:.3f}")
```




```
k=2, silhouette score=0.780
k=3, silhouette score=0.778
k=4, silhouette score=0.695
k=5, silhouette score=0.714
k=6, silhouette score=0.715
```

[67]:
```
# This line of code sets the chosen number of clusters to 3 based on the elbow␣
 ↪and silhouette score analysis.
# Choosing k=3 means the algorithm will try to partition customers into three␣
 ↪distinct segments.
optimal_k = 3

# Initializes a KMeans clustering model with 3 clusters.
# The parameter random_state=42 ensures reproducibility so results stay␣
 ↪consistent across runs.
kmeans = KMeans(n_clusters=optimal_k, random_state=42)

# This line fits the KMeans model on the scaled feature dataset X and␣
 ↪simultaneously predicts cluster labels.
# Each data point is assigned to the cluster whose centroid is closest.
```

```
# The resulting array labels contains a cluster index (0, 1, or 2) for each␣
 ↪customer.
labels = kmeans.fit_predict(X)

# This line adds a new column cluster to customer_features containing each␣
 ↪customer's cluster assignment.
# This permanently associates each customer with their respective segment.
customer_features['cluster'] = labels
```

Evaluation

[68]:
```
# This line calculates the Silhouette Score, which measures how well each data␣
 ↪point fits within its cluster compared to other clusters.
# A score closer to 1 indicates well-separated and compact clusters.
sil_score = silhouette_score(X, labels)

# This line computes the Davies-Bouldin Index, which evaluates average␣
 ↪similarity between clusters.
# Lower values indicate better cluster separation and compactness.
db_index = davies_bouldin_score(X, labels)

# This line prints the computed Silhouette and Davies-Bouldin scores with three␣
 ↪decimal precision.
# This output provides an immediate view of how well the clustering performed.
print(f"Silhouette Score: {sil_score:.3f}, Davies-Bouldin Index: {db_index:.
 ↪3f}")
```

```
Silhouette Score: 0.778, Davies-Bouldin Index: 0.730
```

Visualization & Interpretation

Dimensionality Reduction (PCA)

[69]:
```
# This line creates a PCA (Principal Component Analysis) object to reduce data␣
 ↪to 2 components.
# PCA transforms high-dimensional features into two new dimensions that capture␣
 ↪the most variance.
pca = PCA(n_components=2)

# This fits PCA on the scaled feature dataset X and transforms it into two␣
 ↪principal components.
# It produces a 2D representation of the original multi-feature data.
pca_components = pca.fit_transform(X)

# This adds the first principal component as a new column pca1.
# This represents one axis in the 2D PCA-transformed space.
customer_features['pca1'] = pca_components[:,0]
```

```
# This adds the second principal component as pca2.
# This represents the second axis in the PCA space.
customer_features['pca2'] = pca_components[:,1]

# This line of code creates a new figure for plotting with a specified size of␣
 ↪8x6 inches.
# Larger figures make it easier to view detailed cluster separation.
plt.figure(figsize=(8,6))

# This uses Seaborn to create a scatter plot of customers in PCA space.
# Points are colored by their cluster label, visually showing separation.
# The palette='Set1' provides distinct colors for better visual distinction␣
 ↪between clusters.
sns.scatterplot(x='pca1', y='pca2', hue='cluster', data=customer_features,␣
 ↪palette='Set1')

# This adds a title to the scatter plot.
# It clarifies that the plot visualizes customer segments derived from PCA.
plt.title('Customer Segments (PCA 2D)')

# This renders and displays the plot in your notebook output.
# It ensures the PCA-based cluster visualization is visible.
plt.show()
```



Cluster Interpretation

[70]:
```
# This groups the data by cluster and calculates mean values for each numeric␣
 ↪column.
# It produces an aggregated profile of each cluster.
cluster_summary = customer_features.groupby('cluster').mean()

# This prints the aggregated cluster summary table to the notebook output.
# It shows average values like total spending, number of orders, and basket␣
 ↪size for each cluster.
print(cluster_summary)
```

```
         num_orders  total_price  unique_categories  total_items  \
cluster
0          1.012453   104.851336           1.000000     1.124590
1          1.025443   714.139965           1.004771     1.569514
2          1.857626   234.065776           2.045205     2.553866

         avg_spend_per_order  avg_basket_size      pca1      pca2
cluster
```

```
0                103.687522          1.109999 -0.307383 -0.070764
1                834.100357          1.519272  4.505199 -1.947115
2                137.636650          1.522021  3.133874  6.271422
```

3 Part C: Market Basket Analysis Using Association Rules

[71]:
```
# This line helps identify the 15 most frequently purchased product categories␣
 ↪from the 'basket' dataframe.
# To do this, 'sum()' was used to add each column in the 'basket' dataframe and␣
 ↪sort them in descending order using 'sort_values()'.
# Only 15 columns were stored in 'top_products' out of 73 columns in 'basket.'
top_products = basket.sum().sort_values(ascending=False).head(15).index

# This creates a new data frame, 'basket_top' from the columns stored in␣
 ↪'top_products.'
# This helps filter the original basket to only focus on the most frequent␣
 ↪product categories that will be used for market basket analysis later.
basket_top = basket[top_products]

# This line filters the 'basket_top' to include only rows or the transactions␣
 ↪that bought at least two (2) of the 15 products.
# The function 'astype(bool)' is to convert these values to boolean as they␣
 ↪were originally integers.
# This is needed as the Apriori algorithm as they mine frequent itemset for␣
 ↪Boolean association rules (Han et al., 2012, pp. 243-278).
basket_top = basket_top[basket_top.sum(axis=1) >= 2].astype(bool)

# This line executes the Apriori algorithm to find frequent itemsets.
# A 'min_support' of 0.001 is set, wherein at least 0.1% of transactions is␣
 ↪considered for an itemset to be frequent.
# In this dataset, co-occurrence is very rare, this is why around 100 orders is␣
 ↪set to be the minimum support.
# The use of 'use_colname=True' is for ensuring the resulting itemsets will␣
 ↪used the product names instead of column indices.
frequent_itemsets = apriori(basket_top, min_support=0.001, use_colnames=True)

# This line prints the total number of frequent itemsets discovered by the␣
 ↪Apriori algorithm.
# 'frequent_itemsets.shape[0]' accesses the number of rows in the␣
 ↪frequent_itemsets DataFrame, where each row represents a unique frequent␣
 ↪itemset.
print("Total Frequent Itemsets:", frequent_itemsets.shape[0])

# This generates association rules from the 'frequent_itemset', and a␣
 ↪min_threshold of 0.1 was set for our dispersed data to capture rules but␣
 ↪still filter noise.
```

```
# Association rule finds the associations among the sets of data␣
 ↪(GeeksforGeeks, 2018).
rules = association_rules(frequent_itemsets, metric='confidence',␣
 ↪min_threshold=0.1)
```

```
Total Frequent Itemsets: 101
```

[72]:
```
# This line initiates a new figure for plotting, and figsize=(10, 6) sets the␣
 ↪dimensions of this figure to 10 inches wide by 6 inches high.
# This provides a suitable canvas for the upcoming scatter plot, ensuring good␣
 ↪visibility of the data points.
plt.figure(figsize=(10, 6))

# This uses Seaborn to generate a scatter plot to visualize the relationship␣
 ↪between 'support' and 'confidence' of the association rules.
# The size and hue of the points are mapped to the 'lift' metric, allowing for␣
 ↪a visual representation of the strength and interestingness of each rule,␣
 ↪with point sizes ranging from 30 to 300 and a 'coolwarm' color palette.
sns.scatterplot(data=rules,
                x='support',
                y='confidence',
                size='lift',
                hue='lift',
                sizes=(30, 300),
                palette='coolwarm')

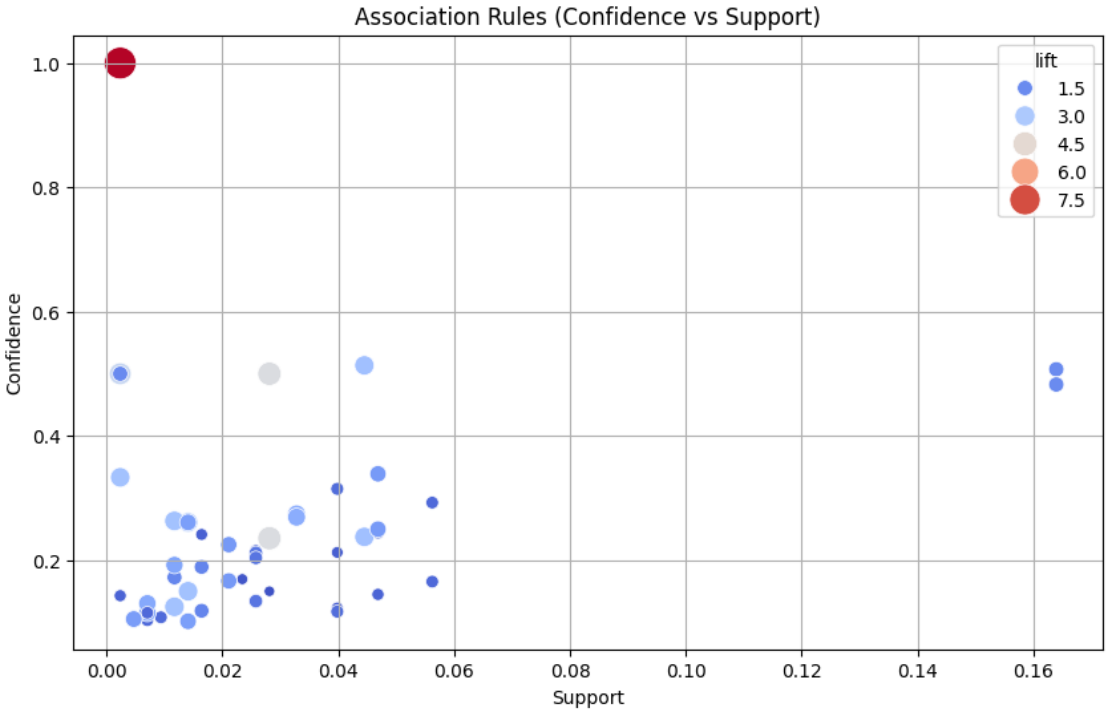
# This line sets the title of the scatter plot to "Association Rules␣
 ↪(Confidence vs Support)".
# This title clearly indicates what the plot represents for context of the␣
 ↪visualization.
plt.title("Association Rules (Confidence vs Support)")

# This line labels the x-axis of the plot as "Support".
# The support metric indicates how frequently an itemset appears in the␣
 ↪dataset, providing context for the horizontal axis (GeeksforGeeks, 2019).
plt.xlabel("Support")

# This line labels the y-axis of the plot as "Confidence".
# The confidence metric measures how often items in the consequent appear given␣
 ↪the presence of items in the antecedent, clarifying the meaning of the␣
 ↪vertical axis (GeeksforGeeks, 2019).
plt.ylabel("Confidence")

# This line adds a grid to the plot background.
# The grid lines enhance readability by making it easier to estimate the values␣
 ↪of data points on both the x and y axes.
plt.grid(True)
```

```
# This line displays the generated scatter plot.
# This renders the plot to make it visible to the user.
plt.show()
```



[73]:
```
# This sorts the 'rules' dataframe in descending order by the lift metric, wherein the top 5 rules are retrieved.
# Only the columns 'antecedents', 'consequents', 'support', 'confidence', 'lift' were selected to view for later.
top_rules = rules.sort_values(by='lift', ascending=False)[['antecedents', 'consequents', 'support', 'confidence', 'lift']].head(5)

# Convert frozensets to strings
# Since the mlxtend stores the antecedents in frozenset, they need to be converted into strings.
# This loops over each frozenset in 'antecedent' and joining them with commas to make them readable.
top_rules['antecedents'] = top_rules['antecedents'].apply(lambda x: ', '.join(list(x)))

# Since the mlxtend stores the consequents in frozenset, they need to be converted into strings.
```

```
# This loops over each frozenset in 'consequents' and joining them with commas␣
 ↪to make them readable.
top_rules['consequents'] = top_rules['consequents'].apply(lambda x: ', '.
 ↪join(list(x)))

# This simply resets the index of the dataframe, with drop preventing the old␣
 ↪index to be added as a new column.
# This helps for a clean and sequential index for the top 5 association rules.
top_rules.reset_index(drop=True, inplace=True)

# This line simply displays the top_rules DataFrame.
# After all the sorting, selection, and formatting, this command outputs the␣
 ↪final table containing the top 5 association rules for analysis and␣
 ↪interpretation.
top_rules
```

[73]:

	antecedents	consequents	support	confidence	lift
0	health_beauty, computers_accessories	sports_leisure	0.002342	1.000000	8.211538
1	health_beauty	perfumery	0.028103	0.235294	4.186275
2	perfumery	health_beauty	0.028103	0.500000	4.186275
3	bed_bath_table, auto	cool_stuff	0.002342	0.500000	3.618644
4	electronics	computers_accessories	0.011710	0.263158	2.809211

4 Part D: Reinforcement Learning: Conceptual Demonstration

Discussion of blue text lines are found in both the notebook and PDF file.

4.1 Research on Reinforcement Learning:

Reinforcement Learning (RL) comes alongside Supervised and Unsupervised learning techniques, which mainly focus on agency through trial and error (GeeksforGeeks, 2018a). This machine learning occurs with an interaction with the environment to reach a goal, receiving feedback of either a reward or a penalty depending on their actions. Through the reinforced feedback, the machine learns over time.

Core Concepts of RL

Term	Description
Agent	Decision-maker performing actions to achieve a goal

Term	Description
Environment	Where the agent operates
State	Current situation of the agent
Action	Possible decisions of the agent
Reward	Feedback based on the action of the agent to guide the agent to a goal
Policy	Maps states into actions; Agent's strategy

4.1.1 *Q-Learning*

This is a reinforcement learning algorithm that learns through updating a table of values (named Q-table). Each entry in the Q-table represents the value of each action in a state (Kovalchuk, 2024). This is an off-policy algorithm that learns from the optimal action developed without the actual choices of the agent.

Q-Learning Process (GeeksforGeeks, 2019a)

1. **Starting with a state**
   In a gridworld environment, the agent starts with a state with its initial location, such as (3, 0). This location is also the agent's state.

2. **Agent selects an action**
   The agent navigates the grid to reach a goal, choosing a move in the grid based on its current state. The Q-table helps in guiding the agent with random choices for new actions or good actions from current Q-values.

3. **Action is done; Environment gives feedback**
   Once an action is done, a new state is given as the result (new location), and feedback is given based on the action (reward). In this example, all moves in the grid give no reward. When the goal is reached, a reward of +10 is given. If a wall is encountered by the agent, a reward of -1 is given, which is a penalty.

4. **Update Q-table; Policy is refined**
   The Q-table is updated from its current run and adjusted based on the received reward and new state. The agent's strategy is refined for choosing an action in each state. This estimates which actions are more beneficial over time.

5. **Cycle repeats**
   The entire process repeats until the optimal policy is developed with the highest possible reward.

4.1.2 *SARSA*

SARSA (State-Action-Reward-State-Action) learns from its knowledge based on the actions the agent takes. In updating a rule, an immediate reward is received, then a future reward is expected. This is an on-policy algorithm that learns from the current policy followed and the actual actions of the agent (Amit, 2024).

SARSA Process

1. **Starting with a state**
   In a gridworld environment, the agent starts with a state with its initial location, such as (3, 0). This location is also the agent's state.

2. **Agent selects an action**
   The agent navigates the grid to reach a goal, choosing a move in the grid based on its policy.

3. **Action is done; Environment gives feedback**
   Once an action is done, a new state is given as the result (new location), and feedback is given based on the action (reward). In this example, all moves in the grid give no reward. When the goal is reached, a reward of +10 is given. If a wall is encountered by the agent, a reward of -1 is given, which is a penalty.

4. **Update State-Action Pair; Policy is refined**
   The state-action pair is updated using SARSA. A state and action are then set based on the received reward and the policy. This policy is not guaranteed to be the best possible action. The agent's strategy is refined for choosing an action in each state. This estimates which actions are more beneficial over time.

5. **Cycle repeats**
   The entire process repeats until the optimal policy is developed with the highest possible reward. Over many runs, the policy converges toward better decisions by continuously improving the Q-values based on actual experience.

4.2 Research Question

Explain how RL differs fundamentally from supervised and unsupervised learning. Why isn't it considered an unsupervised technique?

Comparing the Machine Learning Techniques

ML Technique	Description
Supervised	Learns from labelled data
Unsupervised	Finds patterns in unlabelled data
Reinforcement	Learns through interactions

RL is not considered an unsupervised technique as it uses states, actions, and iterations to learn in its environment. Unsupervised learning focuses on finding patterns through grouping or finding similarities between variables. While RL may learn patterns over time, this mainly learns through trial and error with rewards and penalties.

4.2.1 Scenario: Intelligent Shelf Restocking

Scenario Description: An agent learns to optimize shelf restocking
Problem: A store wants to improve how or when products are restocked throughout the day. Fixed schedules of restocking lead to lost sales, wasted space, and employee time.

RL Concept	Application
Agent	A smart stocking system. Decides which items to restock, how many, and when
Environment	Retail Store with shelf space, product type, time of day, customer traffic, and stock levels
State(s)	Current stock levels Time of day Historical sales Day of week Customer Traffic
Action	Restock a product Delay restocking and wait for data Prioritize categories of products
Reward	+10 if shelves are well-stocked and a sale is made -5 if the product goes out of stock when customers want it -2 for overstocking an item that doesn't sell 0 for neutral events (e.g., maintaining a safe stock level)
Policy	Based on states, choose the restocking action that keeps shelves stocked just enough to meet demand without overstocking.

4.3 Conceptual Walkthrough

4.3.1 *RL Conceptual Walkthrough*

1. **State Space (S):**
   The state represents the current condition of the store, such as:
   - Stock levels (e.g., 2 bottles of water left)
   - Time of day (morning, noon, etc.)
   - Day of week (e.g., Monday)
   - Customer traffic (e.g., high, low)
   - Historical sales (e.g., rolling average)

   Example:
   S = (stock_levels = {'water': 2, 'chips': 5}, time = 11:30 AM, day = Monday, traffic = High)
2. **Action Space (A):**
   The agent can take actions like:
   - Restocking a specific product by a certain amount
   - Delaying restocking
   - Prioritizing product categories

   Examples:
   A = Restock('water', 5)

A = Delay
A = Restock('snacks', 10)

3. **Reward Function (R):**
   - +10: shelves are stocked and a sale occurs
   - -5: item is out of stock when needed
   - -2: overstocking an item that doesn't sell
   - 0: safe but neutral stock level (no sale or loss)

4.3.2 *RL Learning Process*

1. **Starting with a state**
   Agent begins in a known state:
   S = (stock_levels = {'water': 2, 'chips': 5}, time = 11:30 AM, day = Monday, traffic = High)
   This will serve as the current condition of the environment and basis for the next decision of the agent.
2. **Agent selects an action**
   Based on the current state, the agent selects an action such as restocking a specific item, delaying restocking, or prioritizing a category. Initially, choices may be exploratory (random), but over time, they become guided by the learned policy.
3. **Action is done**
   Based on the current state, the agent chooses an action. Actions may be random at first, but over time, actions are chosen based on the agent's policy. These actions could include restocking a specific product by a certain amount, delaying restocking, or prioritizing on a specific category like beverages or snacks.
4. **Environment gives feedback**
   Once the action is done, say, the restocking of 5 bottles of water, the environment changes. Customers play a part by making a purchase or not; with this, the agent receives feedback based on the reward function.
5. **Update state-action value; Policy is refined**
   The agent updates it state-action pair based on the reward and new state. Multiple runs make the agent learn which actions are optimal. The policy is, then, refined to choose the higher reward over time. Examples of these learned actions and refined policies are restocking fast-selling items before peak hours or delaying low-priority items during low traffic.
6. **Cycle repeats**
   The entire process repeats until the optimal policy is developed with the highest possible reward. Over many runs, an optimal policy is developed that balances customer demand, product availability, and shelf space usage, leading to improved store performance.

4.4 Discussion

4.4.1 *Real-World Retail Setting: Benefits*

1. **Dynamic Decision-Making** RL adapts to changing conditions. In the restocking scenario, the agent adjusts its decision based on real-time factors, which leads to smarter restocking compared to relying on fixed restocking schedules. This can even be helpful despite changing customer behavior or other state changes.

2. **Improved Inventory Management and Operational Efficiency**
   In the example, RL reduces the loss of sales, wasted space, and employee time. For example, an agent might learn that demand for bottled water spikes before lunch on weekdays and prioritize restocking then, which optimizes the store's inventory.
   In turn, employees can now focus on customer service. Over time, the agent learns patterns that humans might overlook, such as subtle seasonal demand shifts, improving long-term efficiency.

3. **Scalability and Personalization**
   Once trained, an RL system can be deployed across multiple branches with similar layouts or inventory. It can also adapt to location-specific behaviors, such as weekend shopping surges in urban areas versus rural ones.

4.4.2 *Real-World Retail Setting: Challenges*

1. **Complexity of Environment**
   The real world is noisy, unpredictable, and sometimes more unique than we thought. Certain scenarios may occur that cannot be accounted for by the RL, and the learned policy may overfit to patterns that don't generalize well.

2. **Initial Data Problems**
   Starting may be a challenge for the RL as it needs interaction for effective learning. In our shelf restocking case, the agent would need to experiment with many restocking decisions and observe outcomes over time. Early in training, poor decisions could be bothersome to train in simulation, or badly affect sales and customer experience in the real world..

3. **Reward Design Complexity**
   A reward function is also difficult to implement, especially with the business rules. For the RL to be given a reward only when sales occur might overlook long-term customer satisfaction or brand loyalty, which are hard to quantify in a simple reward function. A poorly designed reward can lead to unintended behaviors like prioritizing only profitable items regardless of customer needs.

4. **Integration with Existing Systems**
   An RL in a real-world retail store may be costly, as it should be designed to support real-time decision-making. Furthermore, it would need integration with the inventory, POS, and employee schedules.

5. **Safety and Risk Management**
   Considering the early poor decisions of the agent, learning in the real world could possibly

hurt the business or the customers themselves.

5 Part E: Overall Discussion, Conclusion, & Recommendation

Part E is found on separate PDF file.

6 References

- Amit, H. (2024, December 2). SARSA Algorithms Explained - Hey Amit - Medium. Medium. https://medium.com/@heyamit10/sarsa-algorithms-explained-b94078fd658b
- GeeksforGeeks. (2018a, April 25). Reinforcement Learning. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/what-is-reinforcement-learning/
- GeeksforGeeks. (2018b, September 13). Association Rule. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/association-rule/
- GeeksforGeeks. (2019a, February 6). QLearning in Reinforcement Learning. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/q-learning-in-python/
- GeeksforGeeks. (2019b, June 12). Implementing Apriori algorithm in Python. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/implementing-apriori-algorithm-in-python/
- Han, J., Kamber, M., & Pei, J. (2012). Data Mining: Concepts and Techniques. In Data Mining (3rd ed., pp. 243–278). Morgan Kaufmann. https://doi.org/10.1016/b978-0-12-381479-1.00006-x
- Kovalchuk, G. (2024, October 24). A Beginner's Guide to Q-Learning: Understanding with a Simple Gridworld Example. Medium. https://medium.com/@goldengrisha/a-beginners-guide-to-q-learning-understanding-with-a-simple-gridworld-example-2b6736e7e2c9
- Raschka, S. (2014). mlxtend. Rasbt.github.io. https://rasbt.github.io/mlxtend/